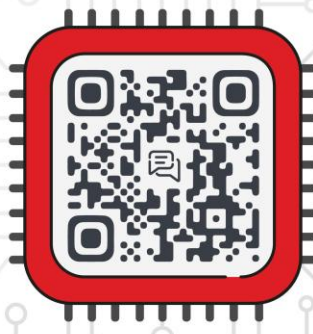




3

17

2040 sp

عملي

كلية الهندسة المعلوماتية

السنة الثالثة

تصميم وحدة التحكم (2) Control Unit

عملي مشترك



RB Informatics; 03/11/2025

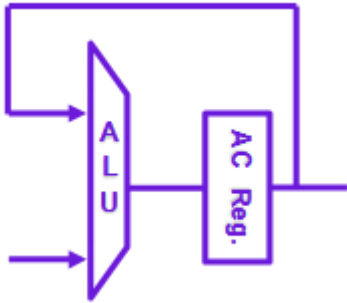
بنيان الحواسيب 2

تذكير من المحاضرة السابقة:

نريد تصميم معالج فيه المواصفات التالية:

عرض ممر المعطيات وكلمة الذاكرة وعرض التعليمية يساوي 16 Bits, خطوط العنوان 12 Bits, والعنونة بالكلمة، وفيه السجلات الآتية:

1. سجل التعليمية (Instruction Register) IR (يحتوي عادةً إما التعليمية (Instruction)، أو رمز التعليمية (Opcode) بعرض 4 Bits)
2. سجل العنوان (Address Register) AR (يحتوي عنوان الذاكرة لأي موقع تتم قراءته أو الكتابة عليه) بعرض 12 Bits
3. سجل المعطيات (Data Register) DR (يحتوي المعطيات التي تتم قراءتها/كتابتها في الذاكرة أو يسمى (MDR/MBR) بعرض 16 Bits)
4. عداد البرنامج (Program Counter) PC (يخزن عنوان التعليمية التالية ليتم جلبها إلى المعالج للتنفيذ) بعرض 12 Bits
5. سجل المراكم (Accumulator) AC (يوصل إلى أحد مداخل وحدة الحساب والمنطق ALU وإلى خرجها) بعرض 16 Bits
6. سجل المستخدم (User Register) R بعرض 16 Bits



التعليمية مؤلفة من الحقلين:

رمز التعليمية Opcode ومعامل التعليمية Operand

يمكن للمعالج تنفيذ التعليمات الثمانية الآتية:

$$1. \text{MOV_R} \Leftrightarrow R \leftarrow -AC$$

نسخ محتوى AC ووضعه في R

$$2. \text{ADD_R} \Leftrightarrow AC \leftarrow -AC + R$$

جمع محتويات R و AC ووضعهما في AC

$$3. \text{AND_D } x \Leftrightarrow AC \leftarrow -M[x] \text{ and } AC$$

أخذ محتوى العنوان المباشر M[x] وتنفيذ and مع AC ووضع الناتج في AC

$$4. LDR_ID\ x \Leftrightarrow R < -M[M[x]]$$

تحميل العنوان الغير مباشر $M[M[x]]$ في R

$$5. STAC_D\ x \Leftrightarrow M[x] < -AC$$

تخزين محتوى السجل AC في العنوان المباشر $M[x]$

$$6. JMP\ x \Leftrightarrow \text{Jump to instruction at address } x$$

القفز إلى التعليمة في العنوان x (قيمة فورية) أي نجعل PC هو x

$$7. JMP_D\ x \Leftrightarrow \text{Jump to instruction (direct address } x)$$

القفز إلى التعليمة في العنوان الموجود في $M[x]$ أي نجلب القيمة ذات العنوان المباشر x من الذاكرة ونضعها

في PC

$$8. NOP \Leftrightarrow \text{No Operation (Do Nothing)}$$

لتضييع الوقت

أكتب مراحل تنفيذ التعليمات الثمانية السابقة بلغة RTL:

ملاحظات:

أولاً: أي تعليمة من التعليمات السابقة ستمر بثلاث تعليمات Fetch التالية

ثانياً: نستخدم لغة RTL فقط عندما يكون لدينا:

- سجل وسجل
- ذاكرة وسجل
- عملية حسابية أو منطقية

ثالثاً: السجلين AR و DR سجلين حارسين للذاكرة بحيث أي عنوان سيمر للذاكرة يكون حصراً عن طريق سجل AR وأي بيانات ستكتب على الذاكرة أو ستقرأ من الذاكرة حصراً أن تمر عن طريق DR

الحل:

مرحلة جلب التعليمة وزيادة العداد PC يتضمن جلب قراءة التعليمات من الذاكرة وحتى حفظها في سجل IR ويمر بثلاث مراحل:

$$1. F1 : AR < -PC$$

وضع السجل PC في السجل AC

$$2. F2 : DR < -M[AR] ; PC < -PC + 1$$

نذهب للعنوان المحتوى في AR ونضع محتواه في DR وعلى التوازي نزيد PC بمقدار 1 (حسب العنونة)

3. $F3 : IR < -DR(op) ; AR < -DR(x)$

نأخذ opcode الموجود في السجل DR ونضعه في السجل IR وعلى التوازي نضع باقي التعليمة operand في السجل AR

1. MOV_R

نضع محتوى السجل AC في السجل R

$$R < -AC$$

عدد مراحل التعليمة هو 4

تعليمة تنفيذ واحدة + ثلاث تعليمات Fetch

2. ADD_R

نجمع محتوى السجل R مع محتوى السجل AC ونضع الناتج في AC

$$AC < -AC + R$$

عدد مراحل التعليمة هو 4

تعليمة تنفيذ واحدة + ثلاث تعليمات Fetch

3. $AND_D\ x$

العنونة مباشرة وبالتالي لدينا تعامل مع الذاكرة وسيتوجب علينا قسم تنفيذ التعليمة إلى مرحلتين

أولاً وضع محتوى العنوان المباشر x في DR

في التعليمة F3 وضعنا بشكل مسبق قيمة العنوان x في AR وبالتالي يصبح القسم الأول من التعليمة

$$AND_D_1 : DR < -M[AR]$$

ثانياً تنفيذ العملية AND بين محتوى DR و AC ووضع الناتج في AC

$$AND_D_2 : AC < -AC \text{ and } DR$$

عدد مراحل التعليمة هو 5

تعليمتي تنفيذ + ثلاث تعليمات Fetch

4. $LDR_ID\ x$

في هذه التعليمة سنذهب لعنوان الذاكرة x ثم نأخذ محتوى x على أنه عنوان آخر في الذاكرة، نذهب إليه ونحمل محتواه

في السجل R سيتم تنفيذ هذه التعليمة على أربع مراحل:

أولاً نذهب للعنوان x المحفوظ بشكل مسبق في AR ونحمل محتوى العنوان في DR

$$LDR_ID_1 : DR < -M[AR]$$

ثانياً نأخذ محتوى الـ DR على أنه عنوان آخر في الذاكرة فنضعه في AR

$$LDR_ID_2 : AR < -DR$$

ثالثاً في هذه المرحلة سنذهب لعنوان الذاكرة المخزن في AR وننقل محتواه إلى DR

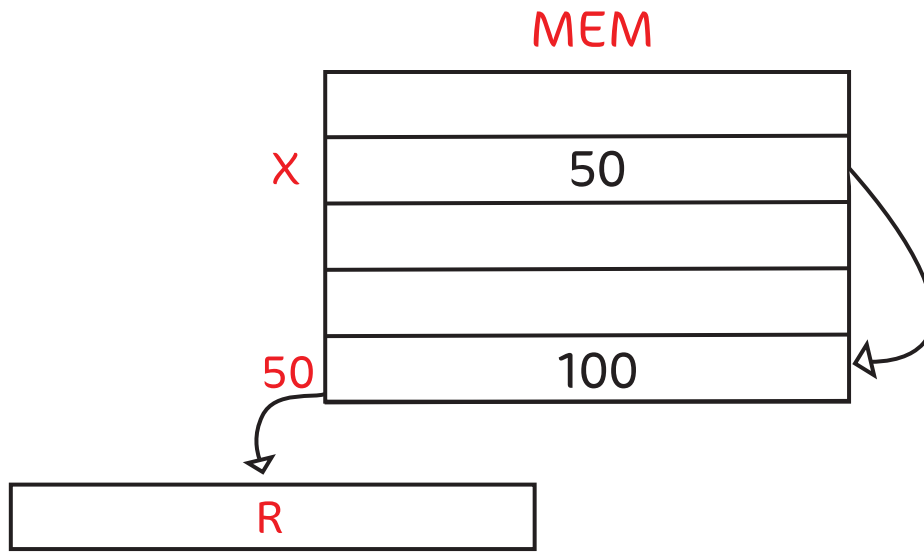
$$LDR_ID_3 : DR < -M[AR]$$

رابعاً سنحمل محتوى DR في R

$$LDR_ID_4 : R < -DR$$

عدد مراحل التعليمة هو 7

رسم للتوضيح:

5. $STAC_D\ x$

تعليمية التخزين تتم على مرحلتين تنفيذ:

$$STAC_D_1: DR < - AC$$

نقل محتوى السجل AC إلى السجل DR

$$STAC_D_2: M[AR] < - DR$$

نحزن محتوى DR في العنوان الموجود في AR

عدد مراحل التعليمية هو 5

تعليمتي تنفيذ + ثلاث تعليمات Fetch

6. $JMP\ x$

القفز يكون من خلال وضع العنوان في PC

$$JMP: PC < - DR(x)$$

عدد مراحل التعليمية هو 4

تعليمية تنفيذ واحدة + ثلاث تعليمات Fetch

7. $JMP_D\ x$

هذه التعليمية يتم تنفيذها على مرحلتين

$$JMP_D_1: DR < - M[AR]$$

سننقل محتوى عنوان الذاكرة الموجود في AR إلى DR

$$JMP_D_2: PC < - DR(x)$$

ثم ننقل محتوى الـ DR إلى PC

عدد مراحل التعليمية هو 5

تعليمتي تنفيذ + ثلاث تعليمات Fetch

8. NOP

تستخدم لتضييع الوقت بالتالي لا يوجد عملية تنفيذ ضمنها

عدد مراحل التعليم هو 4

تعليمه تنفيذ واحدة + ثلاث تعليمات Fetch

بفرض كانت القيمة الابتدائية للسجل PC = 0x28E، ماهي قيم السجلات IR & DR & AR & PC خلال كل مرحلة من مراحل جلب التعليمه الثالثة (باعتبار محتوى الذاكرة الابتدائي المرفق بالجدول)

Address	Content
28A	901E
28B	0A1F
28C	2041
28D	601E
28E	40F8
28F	1020
290	3020
291	611

الحل:

State	PC	AR	IR	DR
-	0x28E	-	-	-
F1: AR <- PC	0x28E	0x28E	-	-

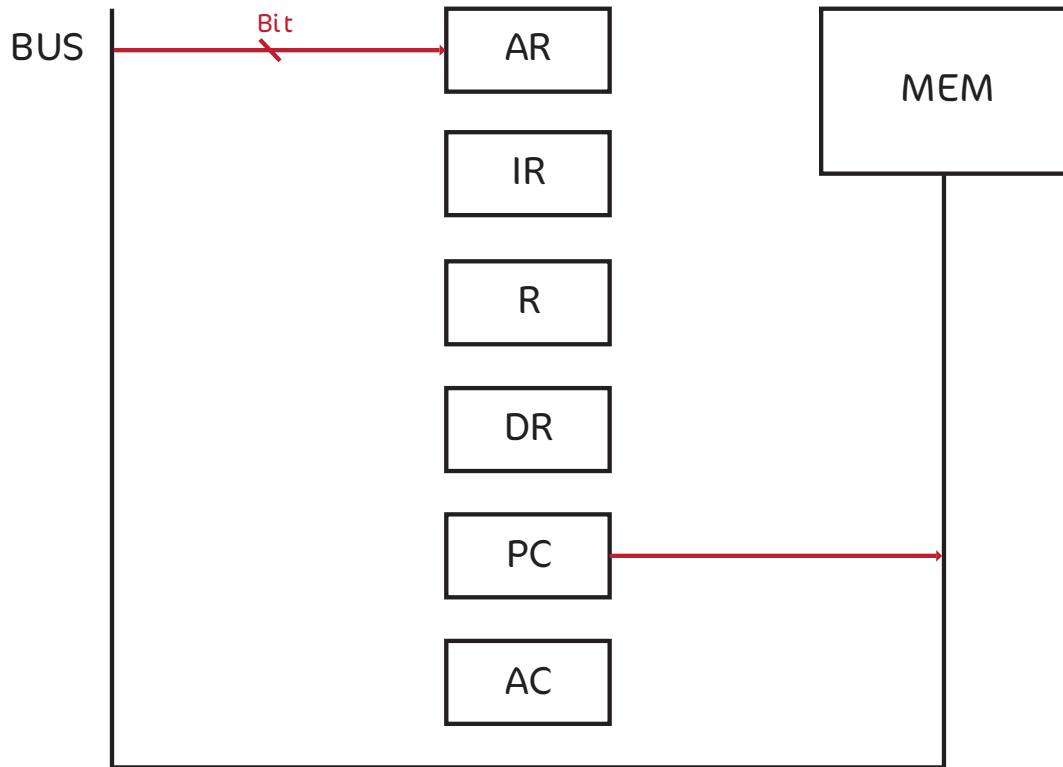
ننسخ محتوى PC إلى AR

F2: DR <- M[AR] PC <- PC + 1	0x28F	0x28E	-	0x40F8
---------------------------------	-------	-------	---	--------

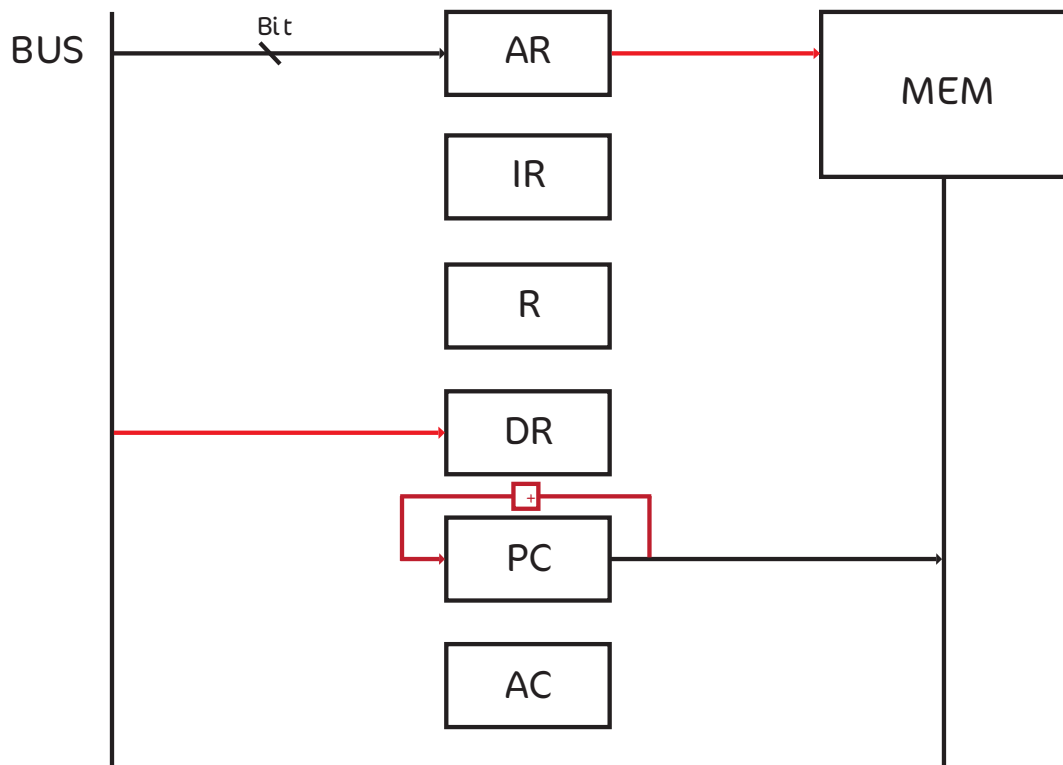
PC <- PC + 1

DR <- M[0x28E]

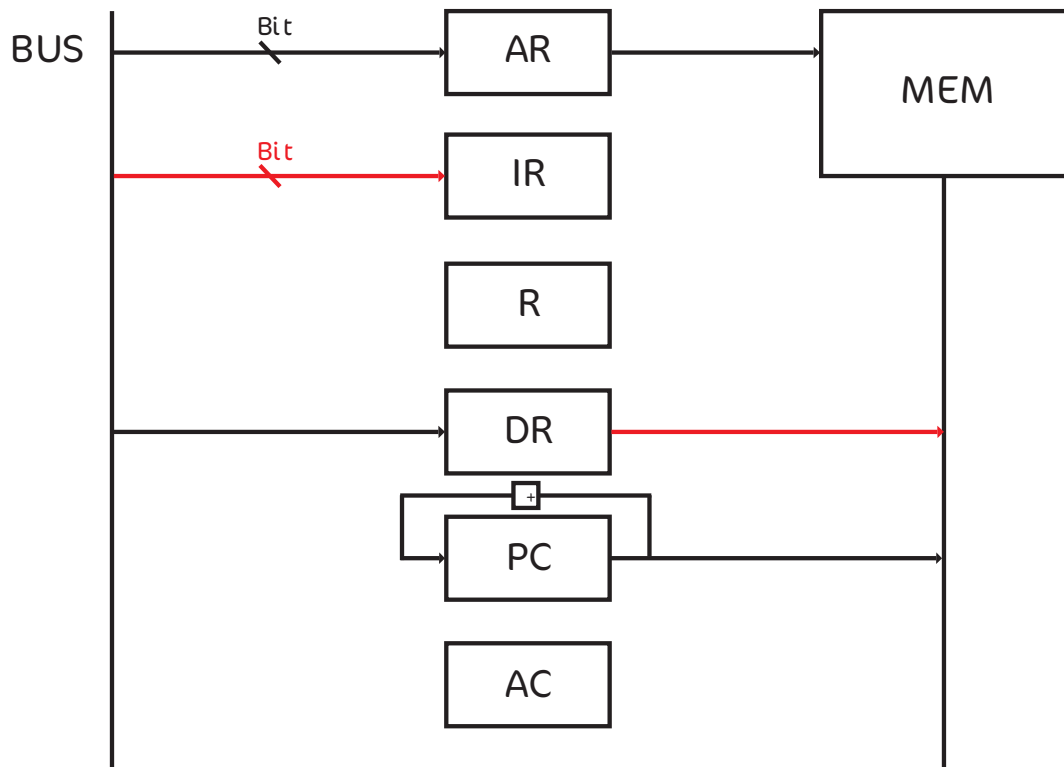
لرسم إشارة $F1: AR \leftarrow PC$ لدينا وصلة من PC إلى BUS ومن BUS إلى AR



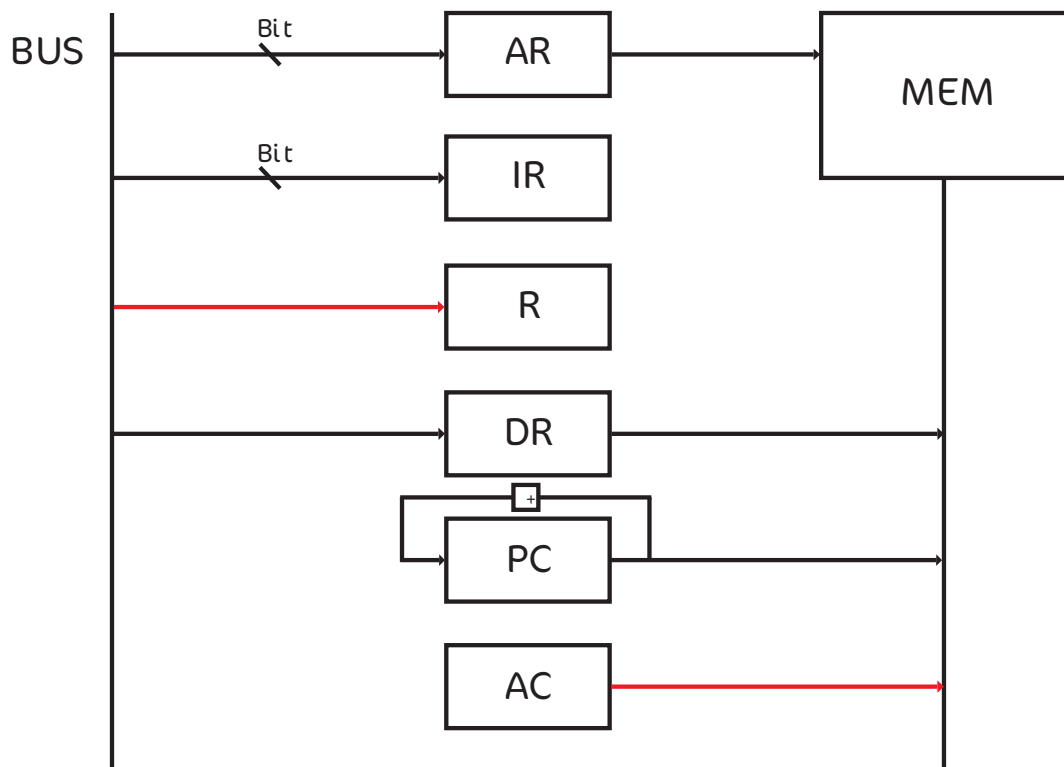
لرسم إشارة $F2: DR \leftarrow M[AR]; PC \leftarrow PC + 1$ لدينا خط من AR إلى M ثم إلى DR ولدينا عند PC وصلة داخلية لا تمر على BUS لزيادة واحد



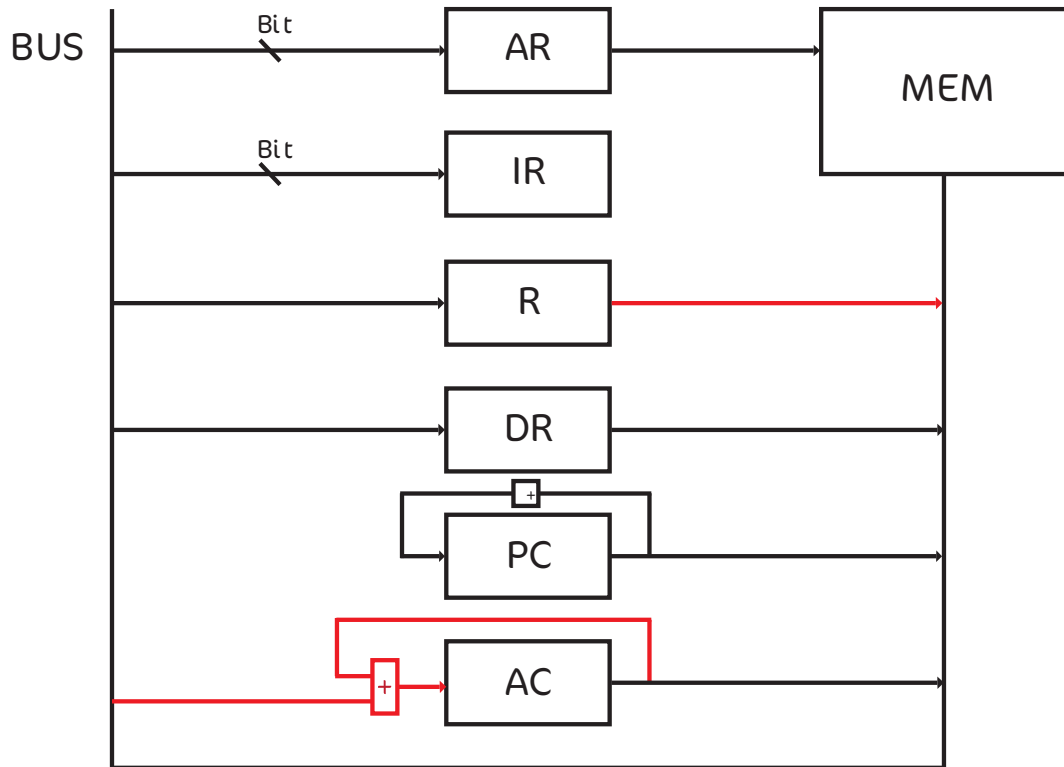
ووصلة من BUS إلى IR ولدينا وصلة من BUS إلى AR



لرسم إشارة $MOV_R: R < -AC$ نحتاج وصلة من AC إلى BUS ووصلة من BUS إلى R



لرسم إشارة $ADD_R: AC \leftarrow -AC + R$ نحتاج إلى وصلة من R إلى AC ووصلة داخلية لAC مع جامع



مرة try ومرة cry وهي
ماشية كده



■ لرسم إشارة AND لدينا مرحلتين

■ المرحلة الأولى:

$$AND_D_1: DR < - M[AR]$$

وصلة من AR إلى M (موجودة سابقاً)

ثم وصلة من M إلى BUS (موجودة سابقاً)

ثم وصلة من BUS إلى DR (موجودة سابقاً)

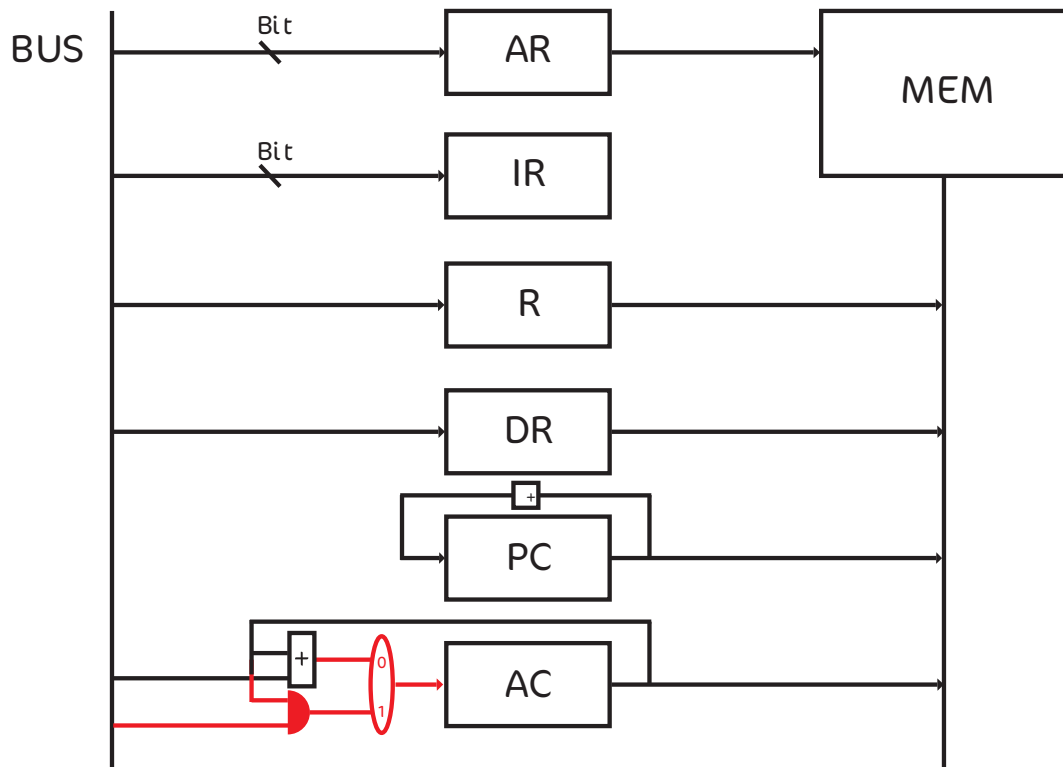
■ المرحلة الثانية:

$$AND_D_2: AC < - AC \text{ and } DR$$

وصلة من DR إلى AC (موجودة سابقاً)

وصلة داخلية إلى AC مع بوابة AND

وهنا نحتاج ناخب لأنه أصبح لدينا جامع وبوابة AND على مدخل السجل AC



- لرسم إشارة $LDR_ID\ x$ لدينا أربع مراحل:
- المرحلة الأولى:

$$LDR_ID_1 : DR < - M[AR]$$

- وصلة من AR إلى M (موجودة سابقاً)
- ثم وصلة من M إلى BUS (موجودة سابقاً)
- ثم وصلة من BUS إلى DR (موجودة سابقاً)
- المرحلة الثانية:

$$LDR_ID_2 : AR < - DR$$

- وصلة من DR إلى BUS (موجودة سابقاً)
- ثم وصلة من BUS إلى AR (موجودة سابقاً)
- المرحلة الثالثة:

$$LDR_ID_3 : DR < - M[AR]$$

مثل المرحلة الأولى

$$LDR_ID_4 : R < - DR$$

- المرحلة الرابعة:

- وصلة من DR إلى BUS (موجودة سابقاً)
- ثم وصلة من BUS إلى R (موجودة سابقاً)

↪ نفس الرسمة السابقة لن نضيف عليها شيء

- لرسم إشارة $STAC_D$ لدينا مرحلتين:
- المرحلة الأولى:

$$STAC_D_1 : DR < - AC$$

- وصلة من AC إلى BUS (موجودة سابقاً)
- ثم وصلة من BUS إلى DR (موجودة سابقاً)
- المرحلة الثانية:

$$STAC_D_2 : M[AR] < - DR$$

- وصلة من DR إلى BUS (موجودة سابقاً)
- ثم وصلة من BUS إلى M (موجودة سابقاً)
- ثم وصلة من AR إلى M (موجودة سابقاً)

↪ نفس الرسمة السابقة لن نضيف عليها شيء

■ لرسم إشارة $JMP\ x$ لدينا مرحلتين:

■ المرحلة الأولى:

■ $JMP_{D_1}: DR < -M[AR]$

وصلة من AR إلى M (موجودة سابقاً)

ثم وصلة من M إلى BUS (موجودة سابقاً)

ثم وصلة من BUS إلى DR (موجودة سابقاً)

■ المرحلة الثانية:

■ $JMP_{D_2}: PC < -DR(x)$

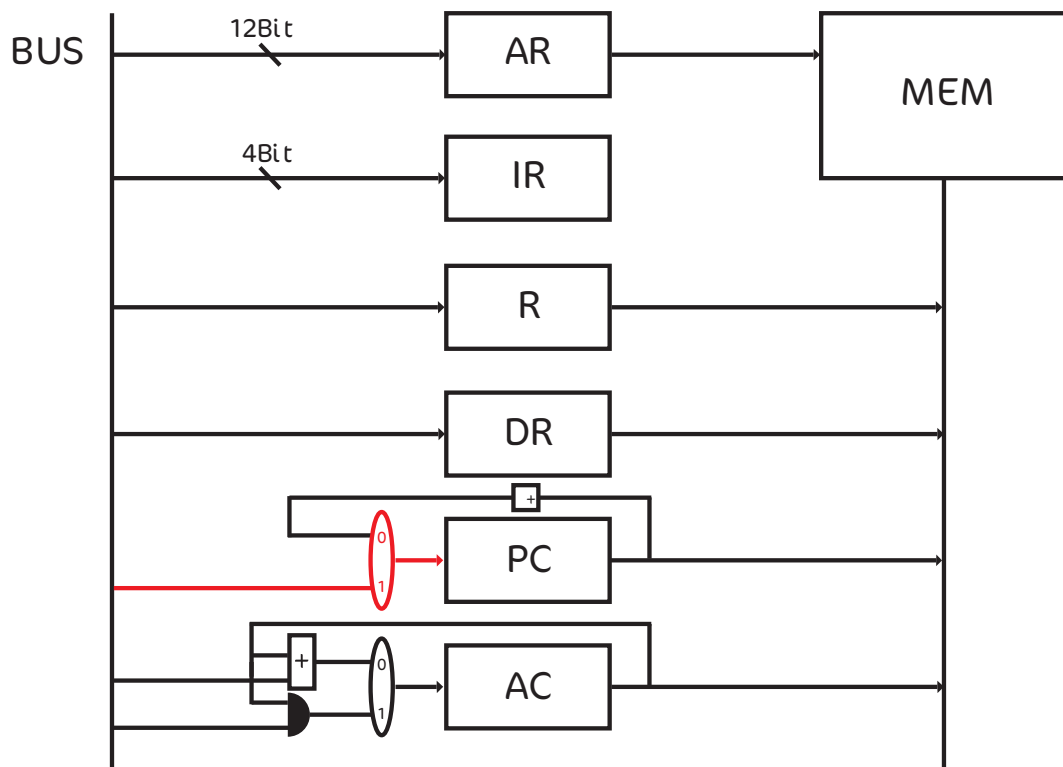
وصلة من DR إلى BUS (موجودة سابقاً)

ثم وصلة من BUS إلى PC

انتبه:

نحتاج ناخب عند مدخل PC

لأنه أصبح لدينا مدخلين الأول من الوصلة الداخلية التي تجمع واحد والثاني من تعليمات $JMP\ x$



هكذا نكون انتهينا من وصل الوصلات لكن لدينا بعض المشاكل علينا حلها:

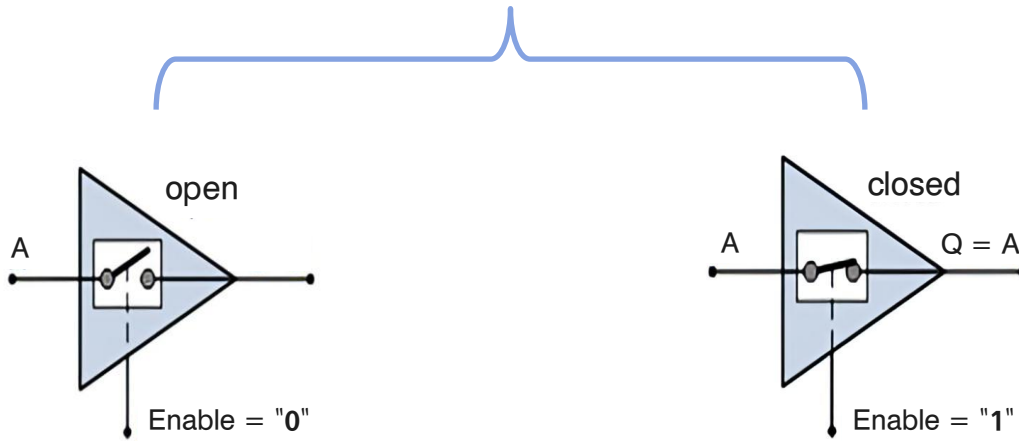
- أكثر من خط يصبون على ال Bus ولتنظيم هذه العملية ومعرفة ما يحتويه ال Bus نستخدم ما يسمى ب Tri-state Buffer.

ال Buffer يعمل بعكس العاكس أي:

- الدخل 0 ← الخرج 0

- الدخل 1 ← الخرج 1

ويحوي ال Buffer خط تحكم إذا كان 0 أي ال Buffer غير مفعّل فيصبح مثل قاطع مفتوح أي لا يمرر قيمة أما إذا كان 1 أي ال Buffer مفعّل ويمرر قيمة:



High impedance

خط التحكم 0،

ال Buffer غير مفعّل ولا يمرر قيمة

Buffer

خط التحكم 1،

ال Buffer مفعّل ويمرر قيمة

- بهذه الحالة عندما نريد قيمة سجل معين نحول ال Buffer الخاص به إلى 1 والباقي 0، بالتالي نضع Tri-state Buffer على كل خط واصل إلى Bus، باستثناء خط AR لأنه موصول إلى Mem وليس إلى Bus وكذلك IR لأنه بالأصل ليس له وصلة إلى Bus.

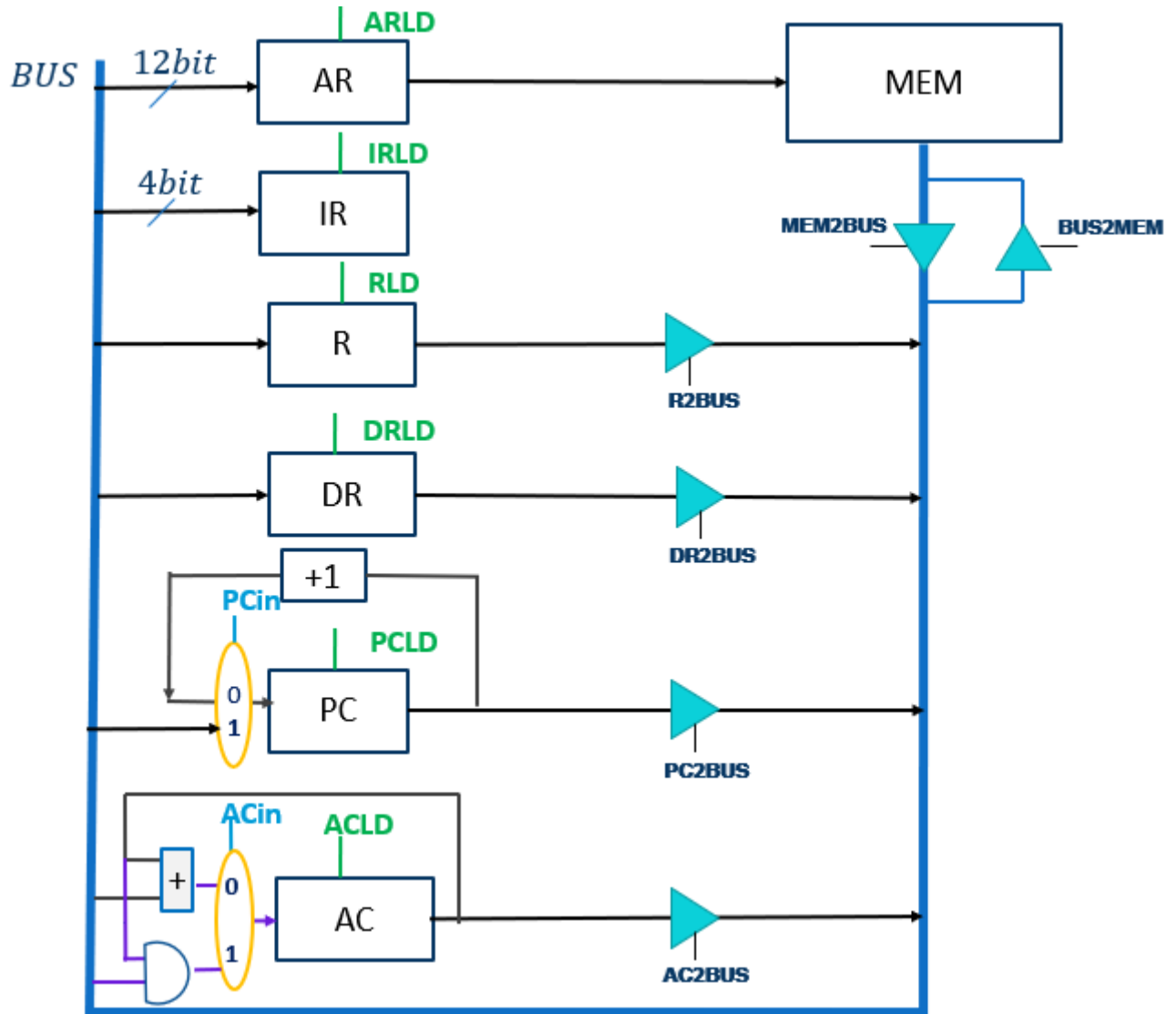
- عندما تخرج قيمة معينة إلى Bus كيف سيحدد إلى أي سجل ستدخل؟

لدينا في السجلات إشارة تحكمية (خط تحكمي) يسمى LD/WR عند الكتابة على السجل تكون قيمته 1 وعند عدم الكتابة تكون قيمته 0.

- كيف سيحدد الناخب ما القيمة التي ستُنتخب ؟

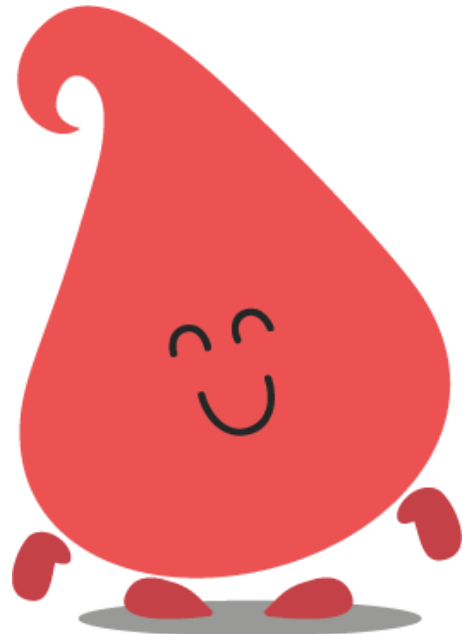
نضيف إشارات تحكمية in لكل ناخب

وبرسم ممر المعطيات بعد إضافة خطوط وإشارات التحكم يصبح الشكل النهائي:



فحصاد اليوم غراس الأمس

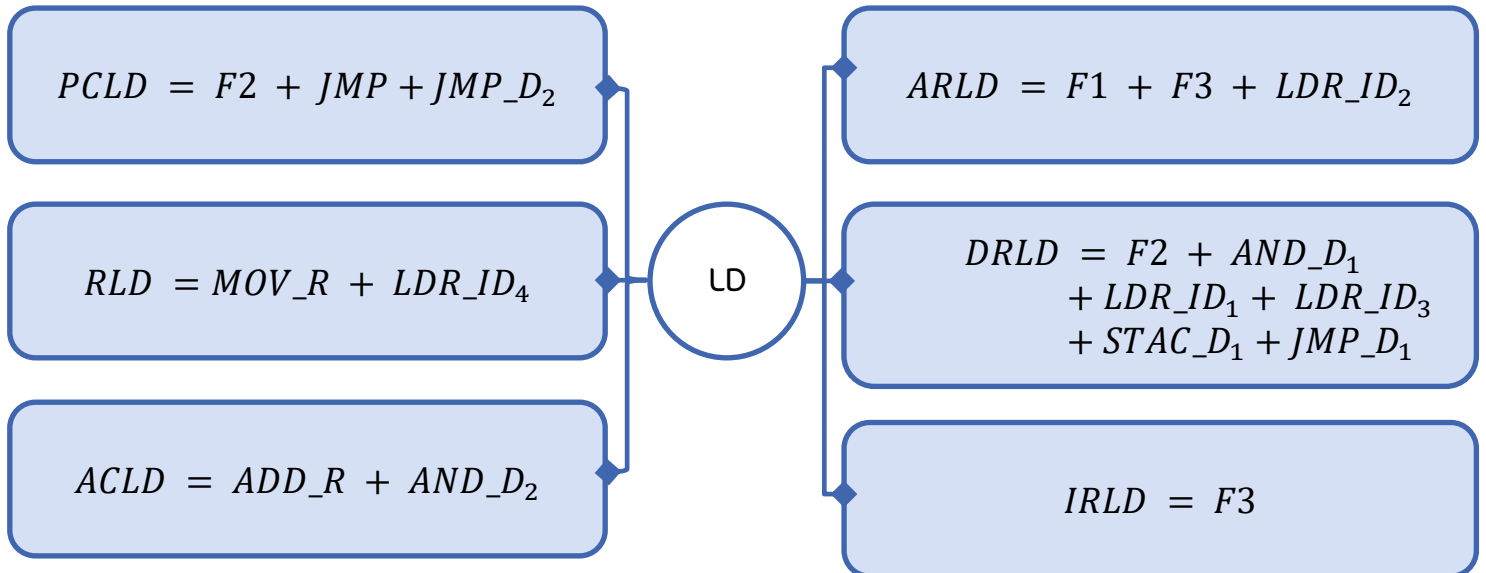
وغرس اليوم غداً يُحصَد



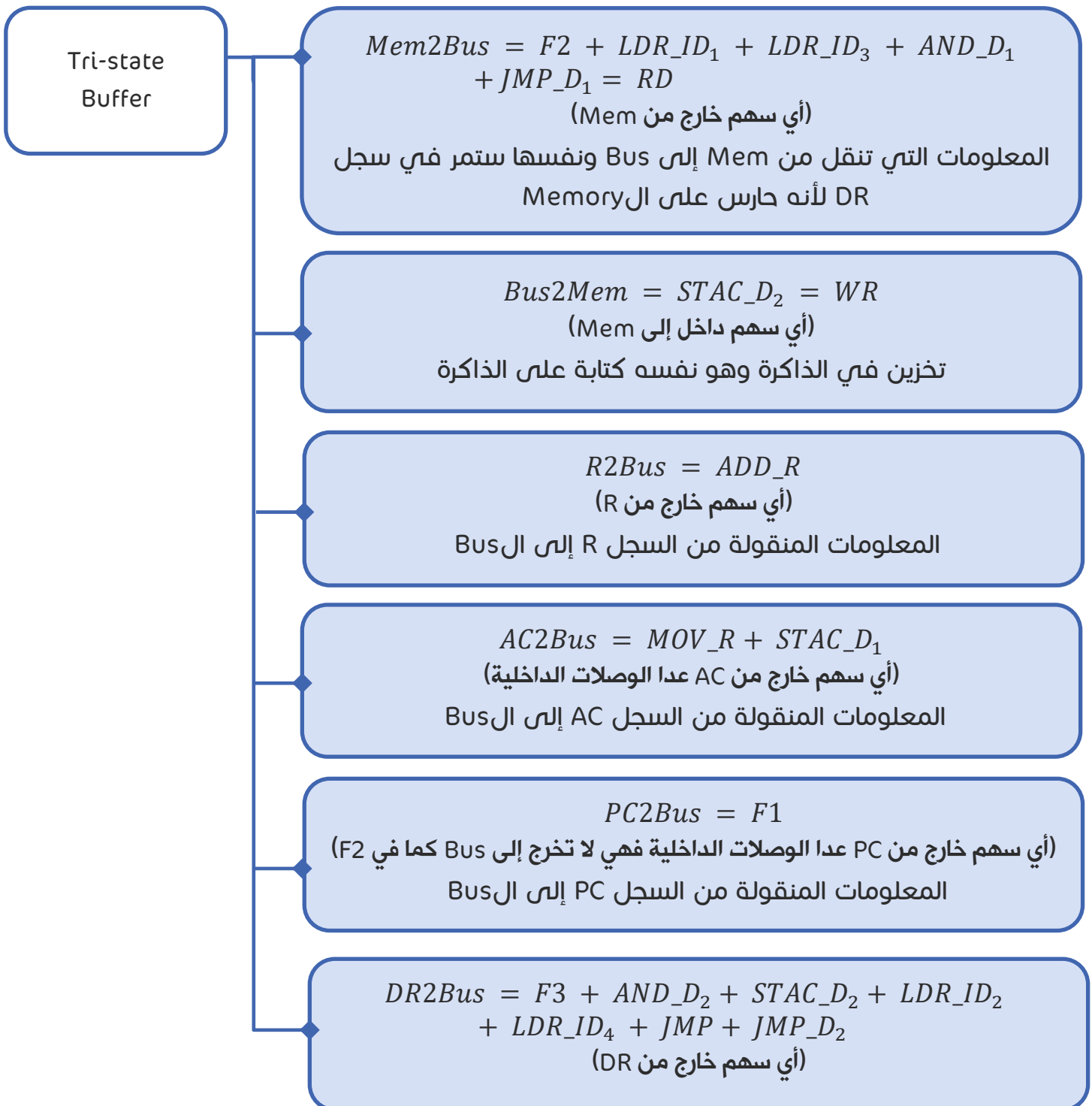
استنتاج معادلات التحكم:

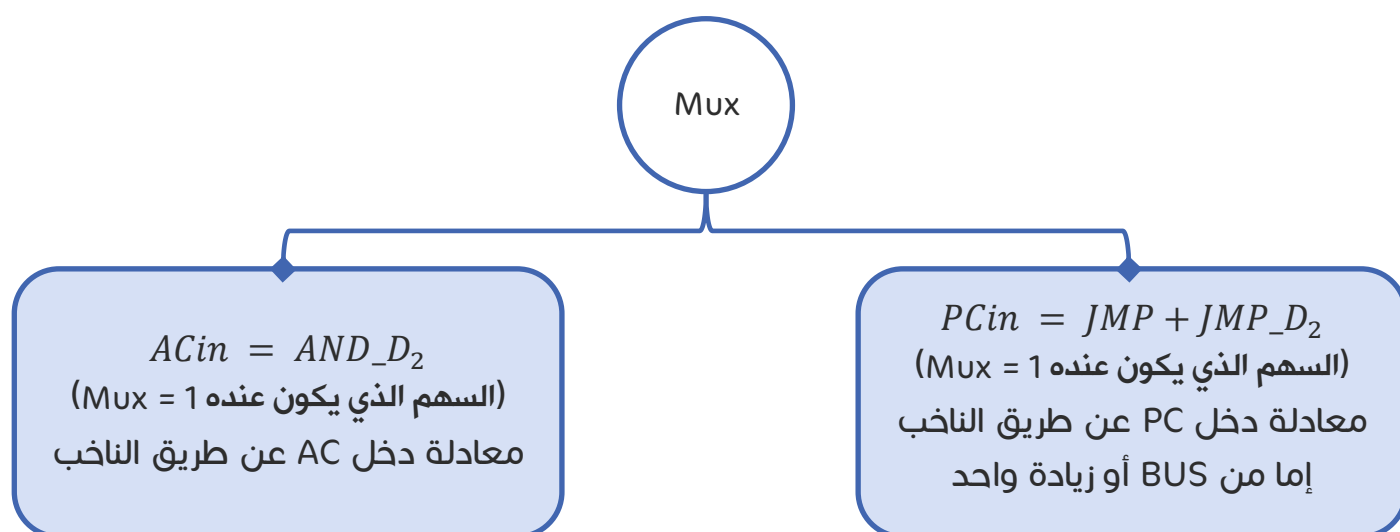
معادلات التحكم تقسم إلى ثلاث أقسام:

1. لاستخراج معادلات LD تكون عبارة عن المراحل التي تكون فيها الكتابة على السجل (أي المرحلة التي يكون فيها السهم داخل إلى السجل)



2. لاستخراج معادلات BUS تكون عبارة عن الحالات التي تكون فيها القراءة فيها من السجل إلى ال Buses (أي المراحل التي يكون فيها السهم خارج من السجل)





من أطلال النوم
سبقه القوم

